HTML API

Shoaib Ghachi

MCT | Technical Trainer | Mentor |SME

Overview

HTML API

-Access and Modify HTML Elements

-Web Storage API

-Get user location

-Fetch API

HTML API

“HTML API" refers to the numerous built-in interfaces in web browsers that use JavaScript to expose data from the browser and computer environment and enable complex functionality within a webpage. These are a crucial part of modern web development, allowing interactions that were not possible in older versions of HTML.

API	Purpose
DOM API	Modify HTML elements
Web Storage API	Store data in browser
Fetch API	Call backend
Geolocation API	Get user location
Form Validation API	Validate inputs
Media API	Control audio/video

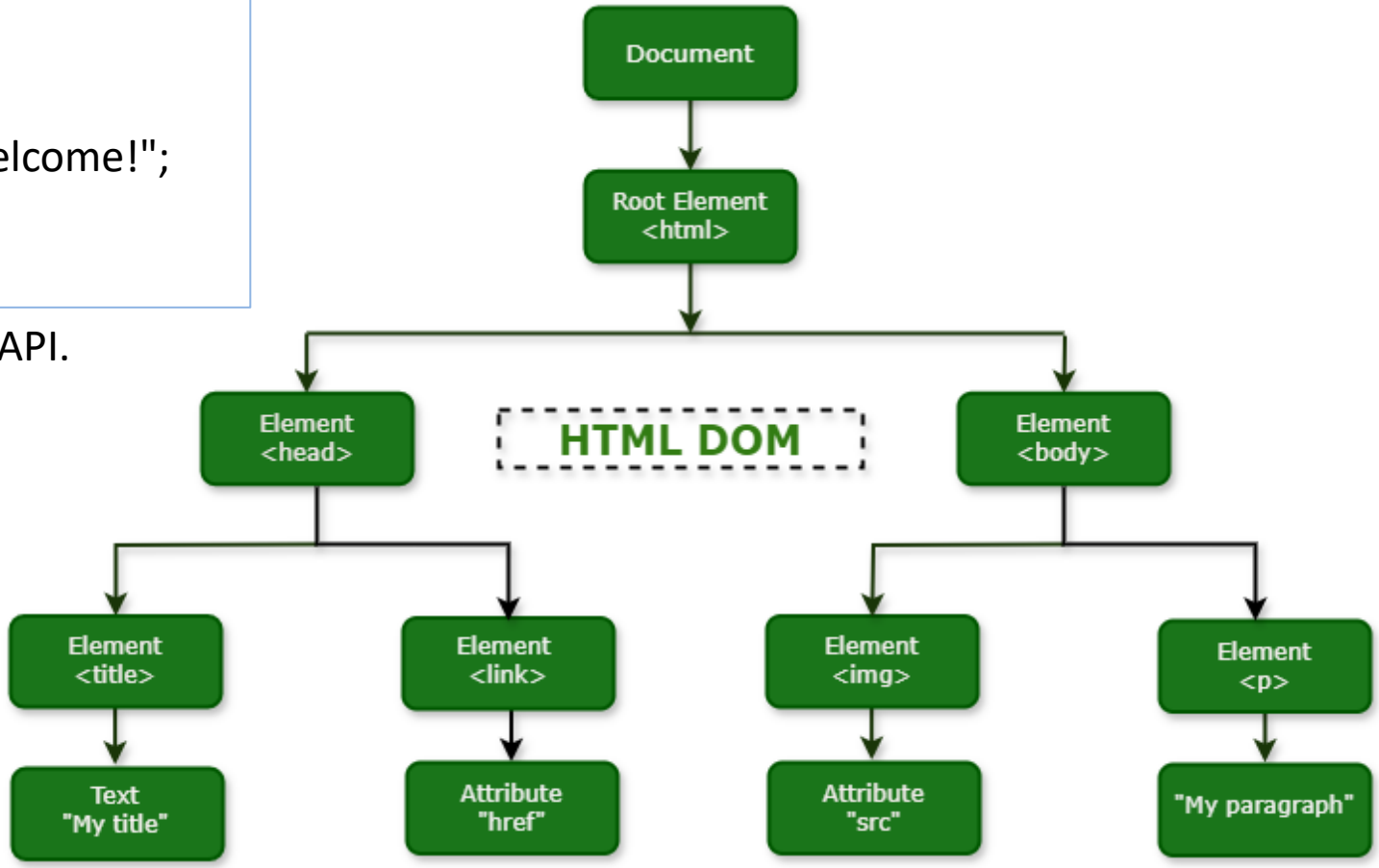
DOM API (Document Object Model)

The DOM API allows JavaScript to access and modify HTML elements.

```
<h1 id="title">Hello</h1>
<button onclick="changeText()">Click</button>

<script>
function changeText() {
  document.getElementById("title").innerText = "Welcome!";
}
</script>
```

Here, JS changes the HTML content using the DOM API.



Web Storage API

The Web Storage API allows browsers to store key–value data on the client side.

It provides two objects:

- **localStorage**
- **sessionStorage**

Both store data in key–value pairs (only strings).

1. localStorage (Permanent Storage):

Data remains even after:

- Browser refresh
- Tab close
- Browser close
- System restart

Until:

- You manually clear it
- Or remove it via code

Real-Time Application :

1.Remember Logged-In User

Imagine you are building a Student Portal. Once user logs in, we store login status.

Use Case: Login persistence across page reloads.

1.Login Page (login.js)

```
function login() {
    var username = document.getElementById("username").value;
    var password = document.getElementById("password").value;

    if (username === "admin" && password === "1234") {
        localStorage.setItem("isLoggedIn", "true");
        localStorage.setItem("username", username);
        window.location.href = "dashboard.html";
    } else {
        alert("Invalid Credentials");
    }
}
```

2.Dashboard Page (dashboard.js)

```
if (localStorage.getItem("isLoggedIn") !== "true") {
    window.location.href = "login.html";
}
```

3.LogOut Button

```
function logout() {
    localStorage.removeItem("isLoggedIn");
    localStorage.removeItem("username");
    window.location.href = "login.html";
}
```

Dark Mode Preference

Even after browser restart → theme remains.

Use Case: Theme persistence across page reloads.

1. script.js

```
// Save Theme
function setDarkMode() {
   document.body.classList.add("dark");
   localStorage.setItem("theme", "dark");
}

// Load Theme
window.onload = function() {
   if (localStorage.getItem("theme") === "dark") {
      document.body.classList.add("dark");
   }
}
```

2. sessionStorage (Temporary Storage)

Data remains even after:

- Until tab is open

Data is removed when:

- Tab is closed
- Browser is closed

Step 1 (Personal Info)

```
function saveStep1() {
   var name = document.getElementById("name").value;
   sessionStorage.setItem("name", name);
   window.location.href = "step2.html";
}
```

Step 2 (Preview Page)

```
window.onload = function() {
   var name = sessionStorage.getItem("name");
   document.getElementById("previewName").innerText = name;
}
```

Difference Between localStorage and sessionStorage

Feature	localStorage	sessionStorage
Data Lifetime	Permanent	Until tab closes
Survives Browser Restart	✅ Yes	❌ No
Scope	Shared across tabs (same origin)	Only same tab
Storage Limit	~5MB	~5MB
Use Case	Login, theme, cart	Form steps, OTP session

When to Use What?

Scenario	Recommended
Remember login	localStorage
Save cart items	localStorage
Temporary OTP validation	sessionStorage
Multi-step form	sessionStorage
Dark mode	localStorage

3. IndexedDb

IndexedDB is a **client-side NoSQL database** built into modern browsers.

It allows web applications to store:

•Large amounts of structured data

•Objects (not just strings)

•Files / Blobs

•Offline data

-It works inside the browser — no server required.

- **Why Do We Need IndexedDB?**

LocalStorage and SessionStorage are limited because:

•They store only strings

•Small storage size (~5MB)

•No advanced querying

IndexedDB solves this by:

✓ Storing structured objects

✓ Supporting transactions

✓ Handling large data

✓ Working offline

- **Key Features of IndexedDB**

1. NoSQL database
2. Object-based storage
3. Asynchronous API
4. Transaction support
5. Large storage capacity
6. Indexed searching
7. Works offline (used in PWAs)

- **Important Concepts in IndexedDB**

1. Database
A container that holds object stores.

indexedDB.open("StudentDB", 1);

2. Object Store (Like Table)

Stores data records.

db.createObjectStore("students", { keyPath: "id" });

3. Key Path (Primary Key)

Used to uniquely identify records.

{ keyPath: "id" }

4. Transaction

All operations must happen inside a transaction.

let transaction = db.transaction(["students"], "readwrite");

5. CRUD Operations

Operation	Method
Insert	store.add()
Update	store.put()
Select	store.get() / getAll()
Delete	store.delete()

- **IndexedDB vs LocalStorage**

Feature	LocalStorage	IndexedDB
Data Type	Strings only	Objects
Storage Limit	~5MB	Much larger
Query Support	No	Yes
Transactions	No	Yes
Best For	Simple data	Complex structured data

- **Basic Working Flow of IndexedDB**

1. Open database
2. Create object store (if first time)
3. Start transaction
4. Perform operation (add, get, delete)
5. Handle success/error